Package 'facereaderconverter'

May 27, 2026

Title Process of FaceReader outputs and emotional episodes

Version 0.2.1

Description Tools to convert FaceReader outputs; Markup of emotional episodes.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Depends R (>= 4.1.0)

Imports dplyr,
hms,
janitor,
readr,
stringr,
tidyselect,
tools,
tibble,
Rcpp,
data.table,
tidyr

LinkingTo Rcpp

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

Config/roxygen2/version 8.0.0

Contents

add_delta_column *Add delta column to coding_df*

Description

Add delta column to coding_df

Usage

```
add_delta_column(coding_df, delta = 0.1, delta_window = 0.1, fps = 30L)
```

Arguments

coding_df	Data frame with columns: id, subject, emotion, frame (or video_time), value
delta	Threshold for both up and down
delta_window	Window size in seconds
fps	Frames per second

Value

coding_df with extra column 'delta' where 1 means up and 0 means down

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
add_delta_column(
    coding_df,
    delta_window = 0.1,
    delta = 0.1,
    fps = 30
 )

## End(Not run)
```

convertFRDirectory *Convert a directory of Facereader TXT to CSV*

Description

Reads all Txt files in a folder and sends them through convertFRFiles.

Usage

```
convertFRDirectory(
  inpath,
  outpath = inpath,
  recursive = TRUE,
  pattern = NULL,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  save_metadata = outpath,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  ...
)
```

Arguments

inpath	Path to an existing .txt file.
outpath	Path to save the csvs to defaults to the inpath
recursive	Bool as to whether to look for all files in directory (TRUE) or just the root folder (FALSE)
pattern	a regex pattern of files to test, if NULL then will look for all txt files
values_as_numeric	Save values as numeric, where applicable
clean_names	returns janitor-style clean names
save_metadata	save the metadata as a csv in the outpath, set to NULL to not save
fail_codes	adds a column with the fail reason, True or False. Column then has 0 for success, 1 for fit_failed, 2 for find_failed
duplicate_timecodes_as_error	throws an error if there are duplicate timecodes, if FALSE then throws warning
...	arguments passed as necessary

Value

Invisibly returns the metadata.

Examples

```
## Not run:
convertFRDirectory(
  inpath="directory_of_txt_files",
  outpath="directory_to_save_csvs_to"
  values_as_numeric = TRUE
)

## End(Not run)
```

`convertFRFiles` *Convert Facereader TXT to CSV*

Description

Reads a .txt file skipping the first 10 lines, guesses the delimiter from the first remaining line, writes a .csv with the same basename in the same directory, and returns the path to the created CSV (invisibly).

Usage

```
convertFRFiles(
  inpath,
  outpath = paste0(tools::file_path_sans_ext(inpath), ".csv"),
  return_data = FALSE,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  ...
)
```

Arguments

`inpath`	Path to an existing .txt file.
`outpath`	Path to save the csv to defaults to the inpath
`return_data`	Bool to return the data from the txt rather than the metadata
`values_as_numeric`	Save values as numeric, where applicable
`clean_names`	returns janitor-style clean names
`fail_codes`	adds a column with the fail reason, True or False. Column then has 0 for success, 1 for fit_failed, 2 for find_failed
`duplicate_timecodes_as_error`	throws an error if there are duplicate timecodes, if FALSE then throws warning
`...`	arguments passed as necessary

Value

Invisibly returns the metadata.

Examples

```
## Not run:
convertFRFiles(
  inpath="FaceReaderOutput.txt",
  values_as_numeric = TRUE
)

## End(Not run)
```

convert_to_episodes *Convert coding_df to episodes*

Description

Convert coding_df to episodes

Usage

```
convert_to_episodes(
  coding_df,
  T_up = 0.2,
  T_down = 0.1,
  delta = 0.1,
  delta_window = 0.1,
  min_dur_sec = 0.1,
  consecutive_missing = 150L,
  fps = 30L
)
```

Arguments

coding_df	a dataframe or otherwise from a FaceReader output. id and subject should be present.
T_up	numeric Upper threshold for entering an episode. Default: 0.2.
T_down	numeric Lower threshold for exiting an episode. Default: 0.1.
delta	numeric Minimum k-step change required, computed as the current value minus the value k frames earlier. Default: 0.1.
delta_window	numeric Time window (in seconds) used to derive k, the lag for the k-step difference rule. Default: 0.1.
min_dur_sec	numeric Minimum episode duration (in seconds). Default: 0.1.
consecutive_missing	integer Maximum allowed consecutive missing (NA) frames while in-state before forcing episode end. Default: 150L.
fps	integer Frames per second (sampling rate of the data). Default: 30L.

Details

The function uses the exported native binding hysteresis_state if available; otherwise it will error. It relies on **data.table** for fast grouping and joins.

Value

A list with two elements:

episodes data.table of detected episodes with columns start_frame, end_frame, n_frames, duration_s, id, subject, emotion, and run_id.

coding Annotated data.table containing the original columns plus id, subject, emotion, value, run_id, status, and in_state. status marks episode boundaries with 1L at the start frame and 0L at the end frame; in_state is TRUE for frames inside detected episodes.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
convert_to_episodes(coding_df)

## End(Not run)
```

`map_paths` *Map files from an input root to an output root, preserve structure, create dirs.*

Description

Map files from an input root to an output root, preserve structure, create dirs.

Usage

```
map_paths(input_dir, output_dir, files)
```

Arguments

`input_dir`	Character scalar. The source root directory.
`output_dir`	Character scalar. The destination root directory.
`files`	Character vector. File paths under input_dir to be remapped.

Value

`Character vector of output file paths (same length as files).`

`parse_time_to_frame` *Parse video time to frame index*

Description

Convert a time string into a frame index given frames-per-second (fps). Supports "HH:MM:SS", "MM:SS", or plain seconds and is vectorised over `video_time`.

Usage

```
parse_time_to_frame(video_time, fps)
```

Arguments

`video_time`	Character or numeric. Time strings in "HH:MM:SS", "MM:SS", or numeric seconds.
`fps`	Numeric. Frames per second.

Value

Integer vector. Frame indices computed as round(seconds * fps).

Examples

```
parse_time_to_frame("00:01:23.5", fps = 30)
parse_time_to_frame("1:23.5", fps = 30)
parse_time_to_frame("83.5", fps = 30)
```

Index